

Hochschule für Technik und Wirtschaft Berlin

Fachbereich 4
Campus Wilhelminenhof
Wilhelminenhofstraße 75A
12459 Berlin
<https://www.f4.htw-berlin.de/>



**Hochschule für Technik
und Wirtschaft Berlin**

University of Applied Sciences

Independent Coursework

Simulation of reprojection techniques as shader in Unity

Ruslan Novikov

Matriculation Number: 566919
20.09.2023

Supervised by
Prof. Dr.-Ing. David Strippgen

Hereby I declare that I wrote this document myself with the help of no more than the mentioned literature and auxiliary means.

Berlin, 20.09.2023

.....
(*Ruslan Novikov*)

Abstract

Major companies in the computer graphics industry, such as Nvidia, AMD, and Intel, have recently prioritized research and development of reconstruction techniques to enhance graphics performance while reducing computational demands. Notable examples of these techniques include Nvidia's DLSS and AMD's FSR. These techniques are capable of upscaling low-resolution renders to higher resolutions, sometimes reaching native resolution quality. Additionally, they possess the capability to generate intermediate frames from a history of previous frames, enhancing the perceived motion smoothness.

In virtual reality applications, reconstruction techniques are also employed but seek a different purpose. Their primary objective is to minimize latency as much as possible and maintain a consistent frame rate. In the VR context, ensuring low latency and frame rate stability is critical, as any delay in user input or frame rate inconsistencies can lead to discomfort.

Considering that issues like input lag and inconsistent frame rates can also detract from the gaming experience in non-VR settings, it's conceivable that techniques like timewarp and spacewarp could be adapted for non-VR games

Hence, my goal is to investigate the feasibility of implementing timewarp and spacewarp techniques in non-VR games to assess whether extending these technologies beyond the confines of virtual reality gaming is a worthwhile pursuit.

Zusammenfassung

Große Unternehmen in der Computergrafikbranche, wie Nvidia, AMD und Intel, haben in jüngster Zeit die Forschung und Entwicklung von Rekonstruktionstechniken priorisiert, um die Leistung der Grafik zu verbessern und dabei gleichzeitig den Rechenaufwand zu reduzieren. Bemerkenswerte Beispiele für diese Techniken sind Nvidias DLSS und AMDs FSR Technologien. Diese sind in der Lage, niedrig aufgelöste Bilder auf höhere Auflösungen zu skalieren und dabei in manchen Fällen sogar die Qualität der nativen Auflösung zu erreichen. Darüber hinaus haben sie die Fähigkeit, Zwischenbilder aus einer Historie früherer Bilder zu generieren, was den wahrgenommenen Bildfluss verbessert.

In VR Anwendungen werden ebenfalls Rekonstruktionstechniken eingesetzt. Ihr Hauptziel besteht jedoch darin, die Latenz so gering wie möglich zu halten und eine gleichbleibende Bildrate aufrechtzuerhalten. Im VR-Kontext ist es entscheidend, eine geringe Latenz und Stabilität der Bildrate sicherzustellen, da Verzögerungen bei der Benutzereingabe oder Bildrateninkonsistenzen zu Unbehagen führen können.

Angesichts der Tatsache, dass Probleme wie Eingabeverzögerung und unregelmäßige Bildraten auch die Spielerfahrung in nicht-VR-Einstellungen beeinträchtigen können, ist es denkbar, dass Techniken wie Timewarp und Spacewarp für nicht-VR-Spiele implementiert werden könnten.

Daher ist mein Ziel, die Machbarkeit der Implementierung von Timewarp- und Spacewarp-Techniken in nicht-VR-Spielen zu untersuchen, um zu bewerten, ob die Erweiterung dieser Technologien über die Grenzen des Virtual-Reality-Gamings hinaus lohnenswert ist.

Contents

List of Figures	xi
1 Introduction	1
1.1 Objective	1
1.2 Scope	1
1.3 Outline	1
2 Fundamentals and Related Work	3
2.1 Reprojection techniques in VR	3
2.1.1 Orientational timewarp	4
2.1.2 Positional timewarp	5
2.1.3 Spacewarp	5
3 Project analysis	7
3.1 Overview	7
3.2 Scene objects and structure	7
3.3 Shader control script	7
3.3.1 Shader options	8
3.3.2 State control and movement	8
3.4 Shader	10
3.4.1 Properties	10
3.4.2 Vertex shader	11
3.4.3 Fragment shader	11
4 Implementation and Refactor	17
4.1 Shader control script	17
4.1.1 Post-process effect renderer	17
4.1.2 Shader options	17
4.1.3 Frame states	18
4.2 Shader	19
4.2.1 Projection methods	20
4.2.2 Reprojection methods	22
5 Conclusion	29
Bibliography	31

List of Figures

2.1	Simplified render timeline with no warping. The vertical lines represent a monitor refresh.	3
2.2	GPU render time exceeds display refresh, thus forces frame 2 and 3 to refer to previous frame	4
2.3	Timewarp warp latest image with "latched" head pose	4
2.4	Motion vectors enable spatiotemporal changes, even with stationary cameras	6
3.1	Orientation timewarp allows you to "look around" an image. Everything beyond the image borders is undefined.	13
3.2	Positional timewarp allows you to "look" and "move around" an image. Areas behind objects are seen as occluded areas.	15
4.1	Image quality pre and post refactor	22
4.2	Comparison of occlusion filling algorithms	24
4.3	Demonstration of spacewarp's detection of spatiotemporal changes. Red areas indicate previous pixel position that are now occluded	27

1 Introduction

1.1 Objective

The purpose of this project is to enhance an existing timewarp demonstration in a non-VR setting. This will involve making script and shader modifications to increase its adaptability for a wider range of testing scenarios. Furthermore, I will introduce recent reprojection techniques to align the project with recent advancements in the field, thus expanding the available testing scenarios.

1.2 Scope

I will conduct an analysis of the scene structure, component scripts, and shader code of the "Async Reprojection" Unity Demo by Comrades Stingers. This demo is openly available and compatible with Unity's 2021 LTS version. [Sti22]

Once potential areas for improvement are identified, I will conduct changes to the scene structure, primarily focusing on the character controller script and the included component script and convert the shader from an opaque shader to a post-process shader. This transformation will improve the scene structure and address existing issues.

The shader itself will undergo structural and form-related improvements. Existing methods will be refactored to fix issues caused by numerical errors and new reprojection techniques will be introduced not currently featured in the demo.

Given the project's scope, I will provide a clear comparison of the differences before and after the refactor. However, I will refrain from conducting an in-depth analysis of the results.

1.3 Outline

This coursework is separated into 5 chapters.

Chapter 2

Explains the fundamental reprojection techniques employed in VR games and provides a general overview of how they are utilized to reduce latency and maintain a stable frame rate.

Chapter 3 Here, an examination of the demo is being conducted on the scene, component script, and shader.

Chapter 4

The enhancements introduced in the demo are outlined in this chapter.

Chapter 5

A initial impression of the results will be presented. I will also outline points of interest where ongoing research can be conducted.

2 Fundamentals and Related Work

In this chapter, the frequently used reprojection methods, "Timewarp" and "Spacewarp," will be explained to provide a better understanding of how these techniques reduce motion-to-photon latency and prevent frame dropping. To effectively convey the idea and concept behind the presented techniques, no detailed implementation details will be provided.

2.1 Reprojection techniques in VR

Reprojection refers to an image processing technique where a target image is projected onto a different camera plane. The goal is to find correspondences between the two camera planes for each pixel.

This makes it possible to display an existing image from a different perspective without rendering a new image from scratch.

In virtual reality (VR), reprojection is used to minimize motion-to-photon latency and extrapolate new images from already-rendered ones.

Let's first examine a simplified render timeline without the use of timewarp:

At the start of each frame, the current position of the headset is sampled to generate a virtual camera based on the sampled information. With this camera, the current image is rendered and then displayed once the screen refresh rate is reached. The resulting motion-to-photon latency consists of the combined CPU time, GPU time, and waiting time for V-Sync. [Ant] 2.1

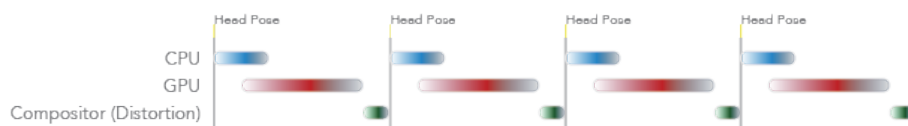


Figure 2.1: Simplified render timeline with no warping. The vertical lines represent a monitor refresh.

Both the CPU and GPU have a limited time frame to generate an image. This time frame corresponds to the interval between screen refresh rates. In cases where the rendering process cannot be completed in time, the last rendered image must be displayed again, effectively doubling the motion-to-photon latency. 2.2

One common and simple way to reduce latency is to disable vertical synchronization, eliminating the wait time after each render and allowing the image to be displayed as soon as it's ready. However, a significant drawback of this method is that disabling vertical

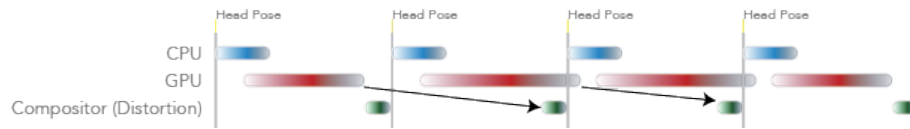


Figure 2.2: GPU render time exceeds display refresh, thus forces frame 2 and 3 to refer to previous frame

synchronization leads to "tearing" artifacts in the image. Furthermore, latency reduction introduced by disabling vertical synchronization is generally marginal and depends solely on the combined processing times of the CPU and GPU.

Timewarp enables the reduction of motion-to-photon latency by executing the warp effect just before the end of the refresh cycle. At the end of GPU rendering, a "latching" operation is performed on the headset's positional data to obtain an updated virtual camera. Subsequently, "Timewarp" can be used to distort the most recently rendered image using the updated virtual camera.

The motion-to-photon latency, with the use of "Timewarp," is the time between "Late-Latching" and the end of the screen refresh rate. The time saved corresponds to the sum of CPU and GPU render times. 2.3

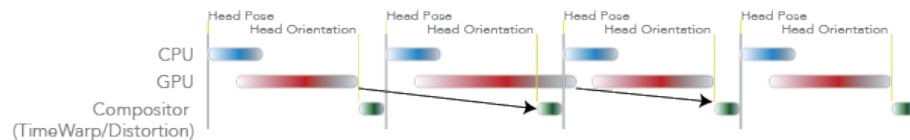


Figure 2.3: Timewarp warp latest image with "latched" head pose

In addition to reducing motion-to-photon latency, "Timewarp" is employed to prevent frame dropping. If the rendering operation takes too long, "Timewarp" and "late latching" are utilized to display the most recently rendered image based on the updated headset information, rather than re-rendering the entire frame.

This capability also allows Timewarp to be used as a method for extrapolating new images. In fact, VR applications intentionally render at only half the hardware target frame rate and extrapolate every other frame. [JZ]

Depending on the difference between the old and new camera planes, different Timewarp techniques are employed. Consequently, Timewarp is generally categorized into two main categories: Orientational Timewarp and Positional Timewarp.

2.1.1 Orientational timewarp

"Orientational Timewarp" only considers the rotation of the headset. This is achieved by mapping the direction vector of each pixel from the current camera plane to the previous camera plane. In simple terms, in Orientational Timewarp, only the current camera's view direction is taken into account, while the position remains the same. This creates an effect where you can "look around" the image.

The main advantage of Orientational Timewarp lies in the simplicity of its algorithm, leading to fast and, importantly, predictable calculation times. When the differences between the current and previous camera positions and orientations are minimal, Orientational Timewarp is sufficient to accurately represent the updated headset information.

However, Orientational Timewarp has the drawback of not accounting for changes in player position. This limitation becomes problematic when significant positional changes occur.

2.1.2 Positional timewarp

Positional Timewarp considers both rotational and positional changes, effectively addressing the previously mentioned limitations. While Orientational Timewarp aligned the view direction with the old camera frame, Positional Timewarp takes a different approach by mapping the depth value of each pixel and projecting it into world space. Subsequently, the projected world position is mapped back to the current camera plane to determine new screen coordinates.

In short, "Positional Timewarp" uses the input image and its depth texture to create a three-dimensional representation that can be viewed from different perspectives and positions.

"Positional Timewarp" effectively addresses the primary limitation of "Orientational Timewarp" but is computationally more intensive and requires more resources. Additionally, "Positional Timewarp" introduces so-called "occlusion holes" – empty areas in the image that arise due to the lack of information in the reprojection process.

These occlusion holes can be mitigated through various hole-filling techniques, that are ideally integrated into the same pass as the reprojection process to minimize additional computational overhead.

Although hole-filling algorithms play a crucial role in reprojection techniques, they are not extensively covered in this work and are only mentioned in passing.

2.1.3 Spacewarp

Both Timewarp techniques yield favorable results in latency reduction, but their inability to detect spatiotemporal changes between frames makes them unsuitable for image extrapolation. When the camera remains unchanged, the output of the Timewarp shader will be identical to its input.

Spacewarp offers an alternative approach to reprojection by utilizing motion vectors. Motion vectors are direction vectors that describe the positional changes of individual pixels from the previous frame to the current frame. With accurate motion vectors, it is possible to calculate the screen position of the previous frame to sample and return the previous color image for each pixel. [ocu]

The primary advantage of motion vectors lies in their ability to detect any positional changes, ensuring precise extrapolation results even when the camera remains stationary. Furthermore, the reprojection calculation becomes remarkably simple and can be implemented in just a few lines of code.



(a) Extrapolation using positional timewarp. Moving objects are not captured



(b) Extrapolation using spacewarp. Moving objects are warped correctly

Figure 2.4: Motion vectors enable spatiotemporal changes, even with stationary cameras

The primary challenge of Spacewarp lies in computing accurate motion vector data. Modern engines can calculate and provide motion vectors for each rendered frame, but Spacewarp is performed in a later stage of rendering when motion vector data is generally not available.

Therefore, earlier versions of Spacewarp used motion estimation to linearly extrapolate motion vectors, utilizing the two previous motion vector textures.

Later iterations of Spacewarp compute motion vectors analytically. This method provides highly accurate motion vectors at any given time, addressing artifacts previously caused by erroneous motion vectors due to incorrect extrapolation.[JZ]

Spacewarp is thus the most precise reprojection technique but also the most computationally and resource-intensive. The additional detection of spatiotemporal changes also leads to more occlusion areas, making the reconstruction of the final image more challenging.

3 Project analysis

3.1 Overview

This chapter introduces the Unity demo of Comrade Stinger. [Sti22] The analysis of the demo aims to develop an understanding of how the project works, in order to better comprehend the changes described in the subsequent chapter.

This chapter includes an examination of the scene structure and the game objects contained within it, the shader script, the available shader options, and the GLSL shader itself.

3.2 Scene objects and structure

The scene consists of an arrangement of simple static and dynamic objects, including a detailed static character model. It also includes a rudimentary first-person perspective controller with a capsule mesh for visualization in the scene, as well as two cameras, "Main Camera" and "Projector."

The scene contains a "Sphere" mesh with a custom material placed directly in the field of view of the "Projector" and "Main Camera" cameras. The material displays the output of the "Timewarp" shader.

The "Main Camera" captures the entire scene, except for the aforementioned sphere mesh. The images captured by the main camera serve as input images for the "Timewarp" shader. The result of the shader pass is applied to a render texture of the material, which is ultimately displayed on the sphere mesh.

The "Projector" camera is responsible for capturing the output of the sphere mesh and its material and displaying it on the screen.

In summary, the "Main Camera" handles capturing the environment, while the "Projector" camera is responsible for displaying the shader output.

3.3 Shader control script

A C# script named "Async Timewarp" controls the movement of the controller, sets all the necessary shader values such as vectors, matrices, and floats that influence the computation of the fragment shader, and initializes all required render textures and buffers. This script is attached to the character controller as a component of type "MonoBehaviour".

3.3.1 Shader options

The user controls the output of the shaders using various options. When starting the application, a GUI displayed on the left side of the window offers the following options:

- Render the game at uncapped FPS: In this mode, the game is rendered without any frame rate limit.
- Freeze rendering new frames and distort using Async Timewarp: This mode freezes the rendering of new frames and applies “timewarp” on the last rendered frame.
- Render the game at a specified FPS: In this mode, the game is rendered at a user-specified frame rate.
- Render the game at a specified FPS and apply “timewarp”: This mode renders the game at a user-specified frame rate but applies timewarp for frame extrapolation

In addition to the four modes, there are two separately selectable options:

- Stretch Timewarp Borders: This option continues the last image color beyond it's borders.
- Include Player Movement in Reprojection : This option incorporates positional changes into the reprojection process, which is equivalent to “positional timewarp”

3.3.2 State control and movement

At the beginning of the script execution, in the Start() method, the render textures for color and depth are instantiated and set as the target buffers for the current "Main Camera." This ensures that when the "Render" function is called, the two buffers are filled with the current camera images. These serve as input to the shader for performing the warping.

In each update call, the state of the selected option is checked. The following example demonstrates the first state, "Render the game at uncapped FPS" as code:

```

1  if (state == 0)
2  {
3      t = 0;
4      setCamera();
5      setFrozenCamera();
6      if (!Cursor.visible)
7      {
8          this.transform.parent.transform.Rotate(Vector3.up, mouseX * sensitivity);
9          this.transform.Rotate(Vector3.right, mouseY * sensitivity, Space.Self);
10         this.transform.parent.Translate(this.transform.parent.forward *
11         MovementY * Time.deltaTime * 5.0f, Space.World);
12         this.transform.parent.Translate(this.transform.parent.right * MovementX
13         * Time.deltaTime * 5.0f, Space.World);

```

```

14     }
15     this.GetComponent<Camera>().Render();
16     accumulatedDT = 0;
17     accumulatedX = 0;
18     accumulatedY = 0;
19     finalDT = Time.deltaTime;
20 }

```

The methods "SetCamera" and "SetFrozenCamera" set the respective camera matrices required for the projection methods. In this case, both methods are executed to ensure that the previous camera plane and the current camera plane are identical. Therefore, when the shader is executed, the output will be identical to the input.

For controller movement, variables "mouseX" and "mouseY" as well as "movementX" and "movementY" are used to capture the mouse and WASD movement key vectors, respectively. Since these vectors refer to the last update call, they would provide incorrect motion values at a simulated frame rate.

Hence, the values of "mouseX" and "mouseY" are accumulated into the variables "accumulatedX" and "accumulatedY" at each update call. For positional changes, the variable "accumulatedDT" is instead summed with the current "deltaTime," which is then used in conjunction with the current "movementX" or "movementY."

In the following example for Option 3, "Render the game at a specified FPS," the accumulated mouse and delta times are used to correct the movement. This directly overrides the character controller's transform after the frame time, noted here as "DeltaTime60," has been exceeded:

```

1  else if (state == 2)
2  {
3      float DeltaTime60 = 1.0f / SlowFPS;
4      t += Time.deltaTime;
5      setCamera();
6      setFrozenCamera();
7      if (t > DeltaTime60)
8      {
9          t -= DeltaTime60;
10         if (!Cursor.visible)
11         {
12             this.transform.parent.transform.Rotate(Vector3.up, accumulatedX * sensitivity);
13             this.transform.Rotate(Vector3.right, accumulatedY * sensitivity,
14             Space.Self);
15             this.transform.parent.Translate(this.transform.parent.forward *
16             MovementY * accumulatedDT * 5.0f, Space.World);
17             this.transform.parent.Translate(this.transform.parent.right *
18             MovementX * accumulatedDT * 5.0f, Space.World);

```

```

19     }
20     this.GetComponent<Camera>().Render();
21     finalDT = accumulatedDT;
22     accumulatedDT = 0;
23     accumulatedX = 0;
24     accumulatedY = 0;
25 }
26 }

```

3.4 Shader

This section describes the operation of the shader, including the shader's properties and the vertex and fragment passes of the shader.

3.4.1 Properties

The shader incorporates the following properties:

Textures

- "_MainTex": Current texture
- "_ColorTex": Input / "Frozen" color texture
- "_DepthTex": Input / "Frozen" depth texture

Vectors

- "_CameraPos": Current camera position
- "_CameraForward": Current camera forward vector
- "_TopLeft": Current direction vector to top left camera far plane
- "_TopRight": Current direction vector to top right camera far plane
- "_BottomLeft": Current direction vector to bottom left camera far plane
- "_BottomRight": Current direction vector to bottom right camera far plane
- "_FrozenCameraPos": Previous camera position
- "_FrozenCameraForward": Previous camera forward vector
- "_FrozenTopLeft": Previous direction vector to top left camera far plane
- "_FrozenTopRight": Previous direction vector to top right camera far plane
- "_FrozenBottomLeft": Previous direction vector to bottom left camera far plane
- "_FrozenBottomRight": Previous direction vector to top right camera far plane

Matrices

- "_WorldToCameraMatrix": Current world to camera or "view" matrix
- "_ProjectionMatrix": Current projection matrix camera
- "_FrozenWorldToCameraMatrix": Previous world to camera or "view" matrix
- "_FrozenProjectionMatrix": Previous projection matrix

Floats

- `"_StretchBorders"`: Controls if color continues beyond image borders. Acts as a boolean.
- `"_ReprojectMovement"`: Controls if movement is considered in reprojection (equivalent to positional timewarp). Acts as a boolean.

3.4.2 Vertex shader

The vertex pass does not introduce significant changes. Its primary purpose is to store vertex information in a structure using the `"v2f"` method, enabling access to this information in the fragment shader.

```

1  struct v2f
2  {
3      float2 uv : TEXCOORD0;
4      UNITY_FOG_COORDS(1)
5      float4 vertex : SV_POSITION;
6      float4 screenPosition : TEXCOORD1;
7      float3 worldPos : TEXCOORD2;
8  };
9
10 v2f vert (appdata v)
11 {
12     v2f o;
13     o.vertex = UnityObjectToClipPos(v.vertex);
14     o.uv = v.uv;
15     o.screenPosition = ComputeScreenPos(o.vertex);
16     o.worldPos = mul(unity_ObjectToWorld, v.vertex);
17     UNITY_TRANSFER_FOG(o,o.vertex);
18     return o;
19 }
```

3.4.3 Fragment shader

The Timewarp algorithm is executed in a single fragment pass. Depending on the setting of the `"_ReprojectMovement"` value, either Orientational Timewarp or Positional Timewarp is performed.

Orientational timewarp

In the case of Orientational Timewarp, the direction vector from the current camera position to the current screen position on the camera plane is determined first. This is achieved by using the direction vectors of the four outermost points previously calculated in the C# shader script using the `"ViewportPointToRay"` method and set as a property.

```

1 DrawTextureMat.SetVector("_TopLeft",
2   cam.ViewportPointToRay(new Vector3(0, 1)).direction);
3 DrawTextureMat.SetVector("_TopRight", cam.ViewportPointToRay(new
4   Vector3(1, 1)).direction);
5 DrawTextureMat.SetVector("_BottomLeft", cam.ViewportPointToRay(new
6   Vector3(0, 0)).direction);
7 DrawTextureMat.SetVector("_BottomRight", cam.ViewportPointToRay(new
8   Vector3(1, 0)).direction);

```

The direction vector is determined by linear interpolation between the four direction vectors, using the current screen position.

```

1 float3 CameraVector =
2   lerp(lerp(_TopLeft, _TopRight, uv.x), lerp(_BottomLeft, _BottomRight, uv.x), 1.0f - uv.y);

```

The viewing direction in the current fragment is added to the previous "frozen" camera position. The resulting world position is then used as input for the "WorldToScreenPos" projection method to calculate the screen position in the previous camera frame. This projection method requires the previous camera projection matrix, view matrix, and camera position.

```

1 float2 uv3 = WorldToScreenPos(_FrozenCameraPos + CameraVector, _FrozenProjectionMatrix,
2   _FrozenWorldToCameraMatrix, _FrozenCameraPos);

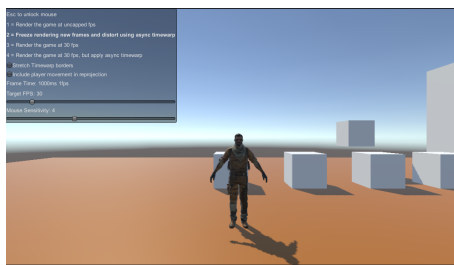
```

The resulting screen position is used to read the color information from the previous frame and return it as the output for the current fragment. Screen positions outside their linear range (0 - 1) are undefined. The options provided here are to either continue the color or use black, controlled by the "_StretchBorder" value, which acts like a boolean in this case.

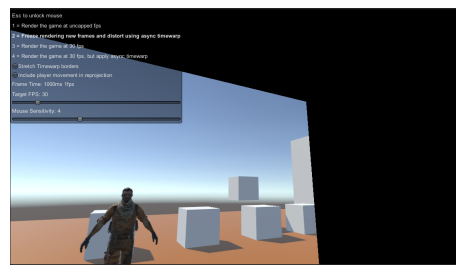
```

1 // sample previous texture at screen pos
2 float3 MainTex = tex2D(_ColorTex, uv3).rgb;
3
4 // if outside linear range and "stretch border" is disabled
5 if ((uv3.x < 0 || uv3.y < 0 || uv3.x > 1 || uv3.y > 1) && _StretchBorders < 0.5)
6 {
7     MainTex = float3(0, 0, 0); // return black
8 }
9
10 return float4(MainTex, 0);

```



(a) Reference image without change in orientation



(b) Reference image with change in orientation

Figure 3.1: Orientation timewarp allows you to "look around" an image. Everything beyond the image borders is undefined.

Positional timewarp

In the case of "Positional Timewarp," a ray marching approach is used. Ray marching attempts to find the nearest world position point for each pixel using the "frozen" depth buffer. In other words, ray marching attempts to determine the nearest hit in a three-dimensional grid of world positions by comparing the depth values of the marched ray position and the reprojected image position of the previous camera.

Ray marching results in three possible outcomes: Hit, Miss, and Occlusion.

A hit occurs when the distance from the marched position to the calculated world position is equal to or less than 0. This indicates that the marched ray has hit the point in the depth buffer and may have even gone beyond it.

In the case of a hit, ray marching is stopped, and the world position is used to calculate the image position from the previous camera frame, which is in turn used to sample the color texture of the previous frame.

A miss occurs when the depth value from the marched position remains greater than the calculated depth from the reprojected image position.

In the case of a miss, the last marched position is used to calculate the image position, similar to a hit situation.

An occlusion situation occurs when marching initially registers a hit, but the distance ultimately exceeds the ray marching step size.

In the case of occlusion, the demo employs the following approach: In a 3x3 kernel, depth values are sampled around the found UV coordinate and compared. The value with the greatest depth is then selected, as it is most likely to sample the background.

```

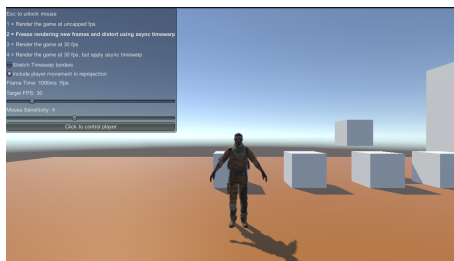
1 // If the pixel was occluded, pick a "nearby" pixel to use instead.
2 // Try returning black if occluded and you will see what it means.
```

```

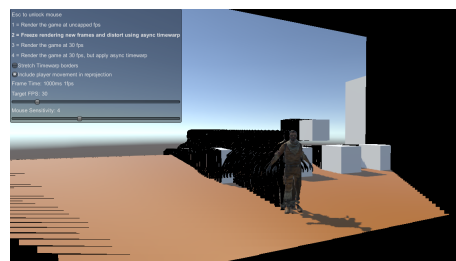
3  // if (occluded)
4  {
5      float2 OffsetUVLeft    = uv3 + float2(1, 0) * 0.01f;
6      float2 OffsetUVRight   = uv3 + float2(0, 1) * 0.01f;
7      float2 OffsetUVTop     = uv3 + float2(-1, 0) * 0.01f;
8      float2 OffsetUVDown    = uv3 + float2(0, -1) * 0.01f;
9
10     float3 MainTexLeft      = tex2D(_ColorTex, OffsetUVLeft ).rgb;
11     float3 MainTexRight     = tex2D(_ColorTex, OffsetUVRight).rgb;
12     float3 MainTexTop       = tex2D(_ColorTex, OffsetUVTop  ).rgb;
13     float3 MainTexDown      = tex2D(_ColorTex, OffsetUVDown ).rgb;
14
15     float Depth              = LinearEyeDepth(tex2D(_DepthTex, uv3).r);
16     float DepthLeft         = LinearEyeDepth(tex2D(_DepthTex, OffsetUVLeft ).r);
17     float DepthRight        = LinearEyeDepth(tex2D(_DepthTex, OffsetUVRight).r);
18     float DepthTop          = LinearEyeDepth(tex2D(_DepthTex, OffsetUVTop  ).r);
19     float DepthDown         = LinearEyeDepth(tex2D(_DepthTex, OffsetUVDown ).r);
20     // Find the furthest away one of these five samples
21     float FurthestDepth = max(max(max(max(Depth, DepthLeft), DepthRight), DepthTop),
22     DepthDown);
23     if (FurthestDepth == DepthLeft)
24         MainTex = MainTexLeft;
25     if (FurthestDepth == DepthRight)
26         MainTex = MainTexRight;
27     if (FurthestDepth == DepthTop)
28         MainTex = MainTexTop;
29     if (FurthestDepth == DepthDown)
30         MainTex = MainTexDown;
31 }

```

In all three cases, ray marching results in an image coordinate used to sample the color information from the previous frame and is returned as the output for the fragment.



(a) Reference image without change in orientation or position



(b) Reference image with change in orientation and position. The player moved slightly left and looks slightly to the right.

Figure 3.2: Positional timewarp allows you to "look" and "move around" an image. Areas behind objects are seen as occluded areas.

4 Implementation and Refactor

This chapter describes the refactoring process and the showcases the implementation of new features. It considers changes made to the shader script, shader options, and the GLSL shader itself.

4.1 Shader control script

The current implementation of the "Async Timewarp" script is responsible for both character control and the shader, making it challenging to use the shader and its shader script in other scenarios without incorporating the predefined character controller. The implementation incorporating two cameras and mesh material could also be simplified, or replaced altogether.

Any code that influenced character control was therefore removed. Additionally, the decision was made to re-implement the shader as a post-process effect, as it can be attached to any camera and thus every kind of control setup. Transforming it into a post-processing effect also eliminates the dependency on the two cameras setup, as the post-processing shader already handles the source and destination of the frames.

4.1.1 Post-process effect renderer

In addition to the changes mentioned earlier, the refactoring primarily consisted of formatting and syntax adjustments. The previous "Async Timewarp" script is no longer a "MonoBehavior" component but instead a sealed class of type "PostProcessEffectRenderer" named "ReprojectionRenderer".

Furthermore, there is a sealed class of type "PostProcessEffectSettings" named "Reprojection," which facilitates input of post-process settings. The configuration of these post-process classes follows Unity's recommendations for post-process effects. [Tec18]

4.1.2 Shader options

The option selection GUI has been replaced by a custom inspector and includes a different setup of available options. The underlying code also handles the transmission of the values to the "PostProcessEffectSettings"

The custom inspector now provides the following options:

Optimization options

- Latency Reduction: This option does not change the framerate but reduces motion-to-latency.
- Frame Extrapolation: This option increases the simulated framerate to the extrapolated framerate.

Reprojection modes

- Orientational Timewarp
- Positional Timewarp
- Spacewarp

Filling modes

- Fill "Out of screen" occlusion: Shows either a blue color or tries to fill occlusion area
- Fill "Depth" occlusion: Shows either a red color or fills occlusion area

Framerate slider

- Simulated Framerate: Sets simulated framerate limit
- Extrapolated Framerate: Sets extrapolated framerate limit

4.1.3 Frame states

In Unity, rendering occurs as frequently as the GPU can handle by default. Enabling V-Sync can limit the number of frames to the screen's refresh rate. To test reprojection at different frame rates, the previous script implemented a simulated frame rate limit. This artificially limits the frame rate by displaying only "simulated" frames on the screen.

In the previous Shader Control script, this was achieved with the following line:

```

1 float DeltaTime60 = 1.0f / SlowFPS;
2 t += Time.deltaTime;
3 setCamera();
4 setFrozenCamera();
5 if (t > DeltaTime60)
6 { ... }
```

If the accumulated time exceeded the frame time, the conditional was executed and the following "Render" method of the camera was executed that would fill the color and depth buffers.

In addition to a simulated frame rate, an extrapolated frame rate limit was implemented. This provides an additional constraint, offering more settings when testing different frame rates, as it was previously have been rendered without limits.

The "EvaluateFrameState" method, found in the C# post-process script, returns an enum that represents the state of the current frame:

```
1 enum FrameState
2 {
3     SimulatedFrame,
4     ExtrapolatedFrame,
5     RegularFrame
6 }
```

The simulated frame refers to the frame generated due to the simulated frame rate. The extrapolated frame refers to the frame generated due to the extrapolated frame rate. The regular frame refers to the frame that doesn't fall into either of these categories. It is called when both simulated and extrapolated frame rates are lower than the frame rate Unity can render at.

4.2 Shader

The shader was reimplemented as a unlit / post-process shader. The majority of the changes were cosmetic and involved simple modifications to form and syntax. The most notable changes were the reduction in properties and the separation of the fragment pass into multiple subpasses.

Textures

- "_MainTex": Source texture
- "_CameraDepthTexture": Current depth texture
- "_CameraMotionVectorsTexture": Current motion vector texture
- "_PreviousColorTexture": Previous color texture
- "_PreviousMotionDepthTexture": Previous motion and depth texture
- "_MotionVectorHistory": History of accumulated motion vectors

Vectors

- "_CameraPosition": Current camera position
- "_PreviousCameraPosition": Previous camera position

Matrices

- "_InvViewMatrix": Current world to camera or "view" matrix
- "_InvProjectionMatrix": Current inverse projection matrix
- "_ProjectionViewMatrix": Current projection-view matrix
- "_PreviousInvViewMatrix": Previous world to camera or "view" matrix
- "_PreviousInvProjectionMatrix": Previous projection matrix
- "_PreviousProjectionViewMatrix": Previous projection-view matrix

Floats

- "_StepSizeFactor": Pos. Timewarp factor that controls ray march step size
- "_MaximumStepSize": Pos. Timewarp value that controls maximum step size.

4.2.1 Projection methods

The previous projection methods were susceptible to rounding errors. These rounding errors resulted in slightly shifted and blurred images as the resulting camera coordinates were not precise. This could be observed by comparing the input image to the output image, even when the camera planes remained unchanged. The expected results are identical images for both input and output, which was not the case, indicating that the methods were computing incorrect coordinates.

The reprojection methods were rewritten and align with more common techniques used in projection pipeline, for transforming coordinates and vectors from one space to another.

The method for converting screen coordinates to world coordinates previously looked like this:

```

1 float3 GetWorldPos(float3 CameraVector, float3 CameraForward, float3 CameraPos, float2 uv)
2 {
3     float d = dot(CameraForward, CameraVector);
4     float SceneDistance = LinearEyeDepth(tex2D(_DepthTex, uv).r) / d;
5     float3 worldPos = CameraPos + CameraVector * SceneDistance;
6     return worldPos;
7 }
```

And has now been replaced with the following method:

```

1 float3 ScreenPosToWorldPos(float2 uv, float3 position,
2 float4x4 invViewMatrix, float4x4 cameraInvProjectionMatrix)
3 {
4     float4 worldSpace = mul(invViewMatrix,
5 mul(cameraInvProjectionMatrix, float4(uv * 2.0f - 1, 1, 1)));
6     return position + (worldSpace.xyz / worldSpace.w);
7 }
```

The new method takes the screen position as its first input parameter, along with the camera position, inverse view matrix, and inverse projection matrix.

First, the screen position is transformed into normalized device coordinates. Then, it is transformed into clip space using the inverse projection matrix. Multiplying by the inverse view matrix results in clip space coordinates, which are then converted into a directional vector in the world coordinate system. This directional vector in the world

coordinate system is added to the camera position to obtain the precise world coordinates.

The other method transforms world coordinate positions into screen coordinates:

```

1 float2 WorldToScreenPos(float3 pos, float4x4 CameraProjection,
2 float4x4 WorldToCamera, float3 CameraPosition)
3 {
4     float nearPlane = _NearClip;
5     float farPlane = _FarClip;
6     float textureWidth = _ScreenParams.x;
7     float textureHeight = _ScreenParams.y;
8     float3 SamplePos = pos;
9
10    SamplePos = normalize(SamplePos - CameraPosition) * (nearPlane
11    + (farPlane - nearPlane)) + CameraPosition;
12    float2 uv = 0;
13    float3 toCam = mul(WorldToCamera, SamplePos);
14    float camPosZ = toCam.z;
15    float height = 2 * camPosZ / CameraProjection._m11;
16    float width = textureWidth / textureHeight * height;
17    uv.x = (toCam.x + width / 2) / width;
18    uv.y = (toCam.y + height / 2) / height;
19    return 1.0f - uv;
20 }
```

The new method takes the world position as its first parameter and additionally requires a projection-view matrix.

```

1 float2 WorldPosToScreenPos(float3 worldCoords, float4x4 projectionViewMatrix)
2 {
3     float4 clipSpacePosition = mul(projectionViewMatrix, float4(worldCoords, 1));
4     float2 uv = clipSpacePosition.xy / clipSpacePosition.w;
5     return uv * 0.5f + 0.5f;
6 }
```

Multiplying by the projection-view matrix is essentially the opposite of the "ScreenPos-ToWorldPos" method shown previously. It transforms the world position into normalized device coordinates. Finally, the normalized device coordinates are converted into screen coordinates.

The previous "WorldToScreenPos" method was causing the error that lead to a diminished output image. Replacing it with the new implementation, resolved the calculation error and improved the image quality significantly. 4.3.

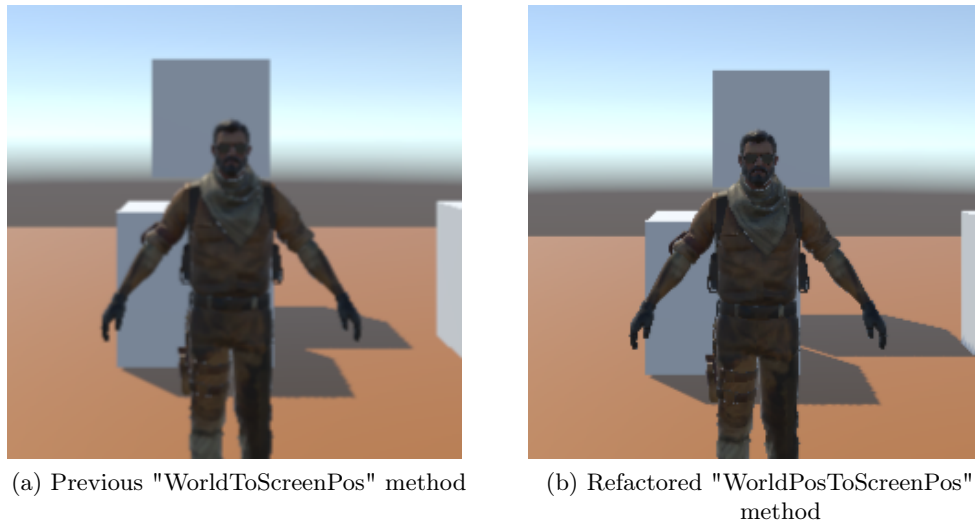


Figure 4.1: Image quality pre and post refactor

4.2.2 Reprojection methods

The previous fragment pass has been divided into multiple subpasses. This change allows for a clear separation of all reprojection methods and also resolved the need for conditional statements in the fragment pass.

The following sections will outline the changes made to the individual reprojection methods and introduce new ones:

Orientation timewarp

Due to the relatively simple algorithm of orientation timewarp, only minor changes have been made. In this case, the refactored orientation timewarp fragment pass uses the two refactored projection methods to calculate the appropriate world, screen coordinates.

Afterwards, as before, the screen coordinate is used to sample the previous color texture, and its color value is returned as the output.

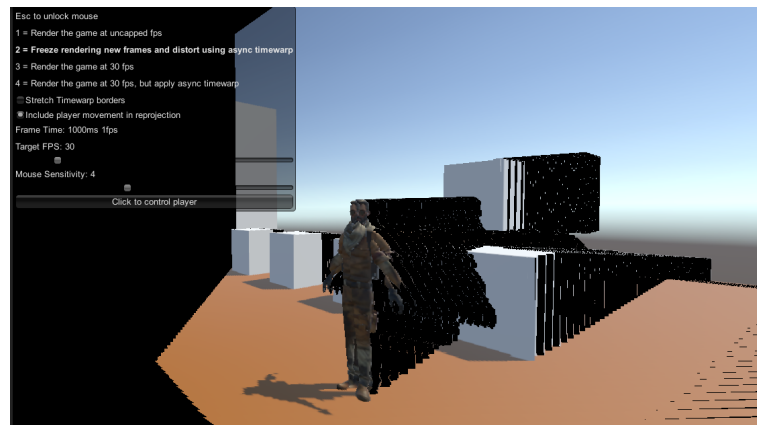
Positional Timewarp

Positional timewarp also remains fundamentally similar, but has been extended with some changes. These changes are based on the paper by Finn Sinclair, where the author proposes a method for positional reprojection based on ray marching previous depth textures. [Sin20] This method is actually very similar to the previous implementation by Comrade Stinger. Though no source has been mentioned, it is reasonable to assume that the previous implementation was based on Sinclairs paper, albeit with some modifications.

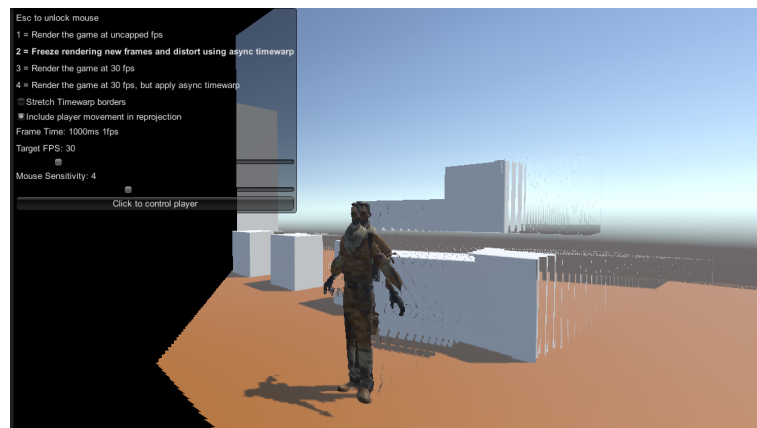
For example, the paper suggests a variable step size with a low step length, whereas the previous implementation used a fixed step size and a higher step length. The variable step size uses the distance between the traversed ray point and the sampled depth point to converge faster towards the intersection point. To avoid overshooting, Sinclair proposes the use of two parameters, alpha and sigma, where alpha multiplies the distance by a factor, while sigma limits the maximum distance. These adjustments limit the maximum ray marching length to 32, whereas it was previously set to 100.

Sinclair proposed algorithm also uses a different approach to solve occlusion areas based on the concept of epipolar lines. In the original implementation, depth values were compared in a square kernel to identify potential areas with a backdrop that could accurately fill the occluded area. This lead to depth value lookup for each occluded pixel. Epipolar lines on the other hand, describe the connecting line between two corresponding image points on two planes. So, when a point is found to be projected onto the previous camera plane, the vector from the starting point of ray marching to the ending point of ray marching already defines one of the epipolar lines. Sinclair thus proposed to backtrack along the epipolar line, as it most-likely will reveal the backdrop of the occluded areas.

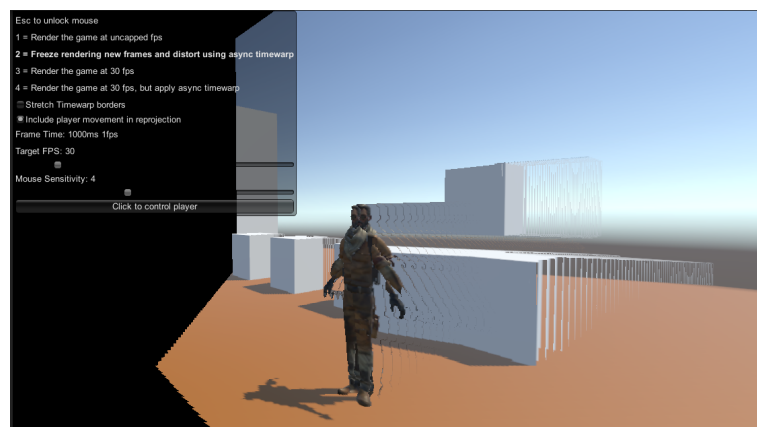
The refactor included both changes that were presented in the paper, which were altered in the previous implementation.



(a) Timewarp with no occlusion filling



(b) Timewarp with 3x3 kernel occlusion filling



(c) Timewarp with epipolar occlusion filling

Figure 4.2: Comparison of occlusion filling algorithms

Spacewarp

Spacewarp was entirely missing in the previous implementation. It was implemented to provide an additional reprojection method, that better reflects the current state-of-the-art.

The spacewarp implementation is based on the "Temporal Reprojection" Unity project by "Keijiro". [git] The project provides a shader with a basic framework for implementing various temporal reprojection techniques, such as motion blur, using motion buffers. It includes algorithms for calculating current and previous screen coordinates and detecting potential occlusion areas depending on depth and motion values. The shader framework has been repurpose to implement the spacewarp algorithm.

One key problem that had to be solved were the provided motion vectors, which were unsuitable due to the simulation of framerate limits. To solve this problem, a history of accumulated motion vectors was used. This texture stored the sum of previously calculated motion vectors from the point of a simulated frame. This was possible because the sum of direction vectors forms direction vectors. Thus, it was possible to access correct motion vectors between simulated frames.

For each simulated frame, the Motion Vector History was reset. This happened in the "ResetMotionVectorHistoryPass." For each regular frame, the "UpdateMotionVectorHistoryPass" was called. In this Fragment Shader Pass, first, the screen coordinate in the previous frame was calculated using the current motion vector. At this coordinate, the value from the Motion Vector History was then read. The read value and the current value were added together and returned as float2 in the current fragment. This value was stored in the Motion Vector History. By storing the accumulated motion vectors in the current screen coordinate, the next call of a regular frame could access them through its current motion vector to obtain a motion vector leading to the original frame and screen coordinate.

The process of the "Spacewarp" algorithm then looked as follows:

First, the current screen coordinates (uv2), as well as the motion vectors (m2) and depth values (d2), were sampled.

```

1 float2 uv2 = i.texcoord;
2 // current motion vector
3 half2 m2 = SAMPLE_TEX2D(_CameraMotionVectorsTexture, uv2);
4 // current depth
5 half d2 = LinearizeDepth(SAMPLE_DEPTH(_CameraDepthTexture, uv2));

```

Afterwards, the screen coordinates of the previous frames were calculated by subtracting the current motion vector from the current screen coordinate (uv1). Using the calculated screen coordinate, the motion vector history is sampled. The variable "m1" contains the motion vector that points to the previously simulated frame.

```

1 // calculate previous screen coordinate

```

```

2 float2 uv1 = uv2 - m2;
3 // sample the combined motion vector
4 half2 m1 = SAMPLE_TEX2D(_MotionVectorHistory, uv1).xy;

```

The sum of the current motion vector and the motion vector present in the history texture is used to obtain the screen coordinates from the simulated frame, here denoted as "cm". The screen coordinate of the simulated frame is calculated by subtracting the accumulated motion vector "cm" by the current screen coordinate.

```

1 // chain current motion vector with previous combined motion vector
2 float2 cm = m1 + m2;
3 // get screen coordinate from init pass
4 half2 uv0 = uv2 - cm;

```

The screen coordinate of the simulated frame is finally used to sample the color and motion/depth information of the previous color and motion/depth textures.

```

1 // sample color and motion/depth texture at init coordinates
2 half4 c0 = SAMPLE_TEX2D(_PreviousColorTexture, uv0);
3 half4 md0 = SAMPLE_TEX2D(_PreviousMotionDepthTexture, uv0);

```

To determine the occlusion areas, the following techniques are used are employed:

- Depth Occlusion Weighting: In this, the depth values between the current depth value and the "frozen" depth value are computed. The greater the difference, the higher the value becomes.
- Out of Screen Testing: This method allows for the identification of areas outside the screen that do not need to be considered in the projection.

Both formulas contribute to detecting occlusion areas in the image and improving the image quality.

```

1 // Depth occlusion testing
2 float docc = abs(1 - d2 / LinearizeDepth(md0.z));

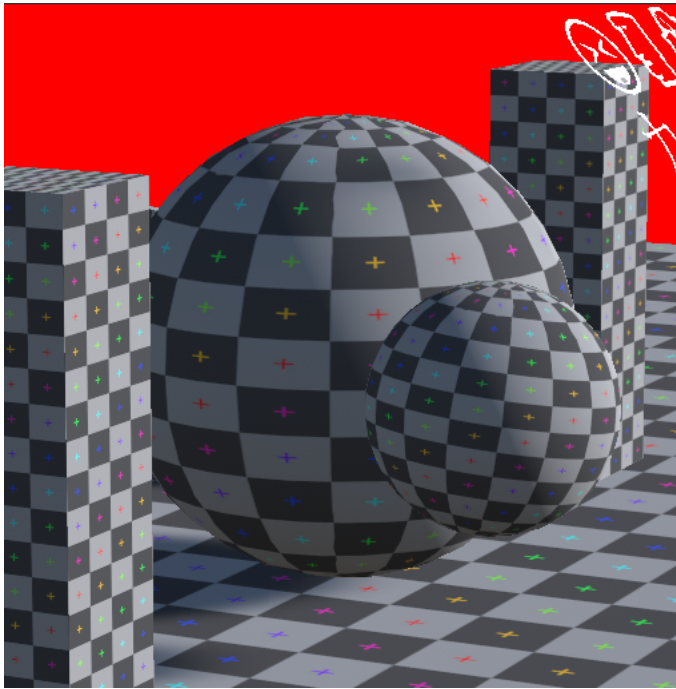
```

```

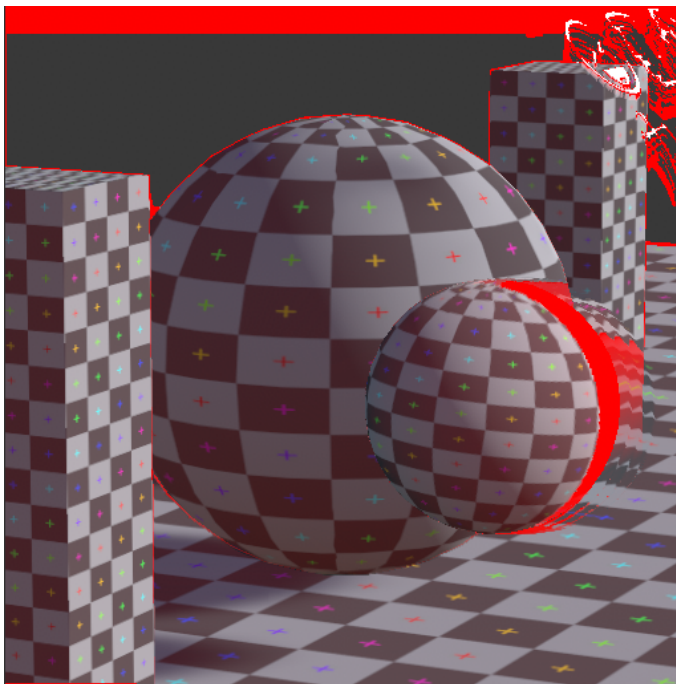
1 // Out of screen test
2 float oscr = any(uv0 < 0) + any(uv0 > 1);

```

The implemented spacewarp technique now provides better extrapolations results, that are especially noticeable when using low simulated framerates.



(a) Position timewarp cannot detect ball movement



(b) Spacewarp detects ball movement

Figure 4.3: Demonstration of spacewarp's detection of spatiotemporal changes. Red areas indicate previous pixel position that are now occluded

5 Conclusion

After refactoring and using Timewarp and Spacewarp in various scenarios, I have come to the conclusion that these techniques have the potential to be employed effectively outside of VR applications. Given that the implementation in different scenarios should now be more straightforward, the next step would be to integrate them into various scenarios like game genres and evaluate the warping techniques.

One significant reason why Timewarp and Spacewarp are utilized in VR is that the movements occurring during VR gameplay are typically quite subtle. In contrast, in non-VR games, players have the freedom to control the mouse and character movements, which likely leads to larger occlusion gaps that need to be filled.

As a result, the primary challenge lies in identifying efficient Hole-Filling algorithms capable of rapidly and reliably filling large gaps. The current algorithms for hole-filling are relatively straightforward and very fast, and they suffice for filling small holes but tend to produce noticeable artifacts when there is a significant difference in camera planes. One potential approach could involve leveraging machine learning methods like Diffusion, even though they may currently be too slow.

Another potential solution is to analytically generate accurate depth textures in real-time, which could greatly enhance the quality of warping by eliminating occlusion holes as a concern. A data structure like ray acceleration structures commonly employed in ray tracing engines could be leveraged to efficiently generate these depth textures.

Bibliography

- [Ant] ANTONOV, MICHAEL: *Asynchronous Timewarp Examined*.
- [git] *GitHub* - *keijiro/TemporalReprojectionExample: Temporal reprojection example for Unity* — *github.com*. <https://github.com/keijiro/TemporalReprojectionExample>. [Accessed 19-09-2023].
- [JZ] JIAN ZHANG, NEEL BEDEKAR, LEONARD TSAI XIANG WEI: *Introducing Application SpaceWarp* — *developer.oculus.com*. <https://developer.oculus.com/blog/introducing-application-spacewarp/>. [Accessed 19-09-2023].
- [ocu] *Application SpaceWarp Developer Guide: Unity | Oculus Developers* — *developer.oculus.com*. <https://developer.oculus.com/documentation/unity/unity-asw/>. [Accessed 19-09-2023].
- [Sin20] SINCLAIR, FINN: *Spatio-temporal reprojection for virtual and augmented reality applications / IDEALS* — *http*. <http://hdl.handle.net/2142/109178>, 2020. [Accessed 19-09-2023].
- [Sti22] STINGER, COMRADE: *Async Reprojection outside of VR*, Oct 2022.
- [Tec18] TECHNOLOGIES, UNITY: *Post-processing Overview*, 2018.